

Managing Customer Involvement in Globally Distributed Agile Projects

Reem Bin-Hezam , Sultan Alyahya
 Information Systems Department
 College of Computer & Information Sciences
 King Saud University
 Riyadh, Saudi Arabia
 rybinhezam@pnu.edu.sa, sualyahya@ksu.edu.sa

Abstract— Agile software projects require frequent interactions between customer and team members especially during requirements analysis and acceptance testing. Though these phases normally require customer availability on-site, this type of involvement might be difficult with globally distributed projects where customer's location is different from team members' location. The traditional methods of customer involvement in distributed agile projects such as using the rich communication technologies (e.g. video-conferencing solutions) are commonly used. The problem with these methods is that team members may not recognize how their technical work requires customer involvement. Even if they know, they might forget to inform the customer or there might be a considerable time between the need for customer involvement and the nearest meeting with him. In addition, the review of the available mechanisms of customer support in thirty-six project management tools reveals a lack of computer-based support for customer involvement. This paper presents a research in-progress which aims to support the effective management of customer involvement in distributed agile teams. It proposes computer-based process support for managing customer involvement which can improve the meaningful communication between customer and agile team and improve customers' awareness with regard to the state of activities in which he participates in.

Keywords—Global Software Development; Project Management; Agile; Process; Tool.

I. INTRODUCTION

Agile software development has been widely adopted and has gained popularity within the last fifteen years. It was introduced by a group of software developers who published a manifesto for software development [1]. In order to meet the core agile values, frequent interactions between customer and team members to understand requirements and fast response to any changes requested by customer are essential.

Effective communication between customer and agile team is a keystone to agile development practices. If this communication is not successful or effective, work will get delayed and productivity will be negatively affected.

Furthermore, the huge expansion of software development industry across the world has encouraged having team members distributed in different geographical areas and sometimes far from customer location.

For the sake of balancing between agility and team distribution, distributed agile teams face many challenges; one challenge is the coordination of customer involvement

[2]. Coordination of customer involvement is one of the primary problems during developing software in agile distributed environments. Unlike traditional software development, one of the main principles of agile methods is to have customers closely involved thorough the whole development process: identifying requirements, prioritizing them and evaluating the output of each project iteration [1, 3]. These processes require frequent face-to-face communications and customer availability through project time. This type of involvement might be difficult with distributed team members, where customer's location is different from team members' location.

To deal with this problem, some studies (e.g. [4]) suggest providing a customer proxy representative who acts as a link between customer and the other team members. However, this solution adds a level of complexity because direct collaboration with customer is very important for the success of software projects [2, 5].

Another solution [6] suggests traveling to the customer site in order to meet the customer and discuss requirements. In addition to the fact that this solution can be costly and not practical for some teams, travelling can only coordinate high level topics with the customer and clearly cannot be sufficient enough to manage the expected frequent communication between customer and development team during the project.

Other studies (e.g. [6]) suggest using rich communication tools, such as video and phone conferencing, as a replacement for the face-to-face meetings. Two problems with this solution are time zone differences and the expense that can come with these tools.

In addition to the limitations of the solutions mentioned above, another common limitation is that team members may not recognize how their technical work requires customer involvement. Even if they do know, they might forget to inform the customer or there might be a considerable time between the need for customer involvement and the nearest meeting with him.

When it comes to putting solutions into practice, researchers found that customer involvement is not needed through the entire project time [4]. Not all involvements and collaborations need to be through synchronous communications or require online availability of the other team members. Only meaningful communication with customers is needed, regardless of how many rich communication channels are available [7, 8]. Most customer involvement and collaboration is needed during

requirement analysis, testing and releasing stages [4]. Therefore, the solutions should focus on supporting activities needed at these stages.

Although there are many project management tools currently available, the review conducted in this research for thirty-six tools shows that current support for most customer involvement processes is missing.

This paper presents a research in-progress which aims to support the effective management of customer involvement in distributed agile teams. It proposes computer-based process support for managing customer involvement which can improve the meaningful communication between customer and agile team and improve customer's awareness with regard to the state of activities in which he participates in.

The remainder of this paper is organized as follows: Section II describes the main customer involvement processes. Section III describes the traditional methods of customer involvement in distributed agile projects whereas section IV provides a review of current tool support. Section V shows how current tools can better support the main customer involvement processes before concluding the paper in section VI.

II. CUSTOMER INVOLVEMENT PROCESSES

Many studies have discussed the points of communications between customer and software team in agile development (e.g. [7, 9-12]). Referring to the different agile practices, we identify below the main customer's communication points through the project lifecycle.

1. Planning and Requirements Analysis

In these stages, the customer involvement is the highest [9]. There is a need to communicate and collaborate with customer to discuss the requirements at the beginning of a project. In this process, requirements are collected, defined, and documented. This may be repeated at the beginning of each iteration and release throughout software development [7].

It is also important to outline the scope of the next release, specify core features to be included, and give an estimated timeline for completion [11]. The main benefit of the planning stage is to make it possible for the project team to visualize the business process that is going to be delivered on the next release. Effective customer collaboration and working closely with them is important at this stage.

The main customer involvement processes associated with the planning and requirement analysis include:

- *Write a user story*: the requirements are written by the customer and the development team breaks them down into smaller tasks.
- *Modify a user story*: if there is a need for modifications, usually after an acceptance test fails, a user story will be modified.
- *Delete a user story*: a user story may be removed if, for example, the customer finds that something is not important or not needed anymore.
- *Prioritize a user story*: the customer chooses the

stories to be included on the next release based on their importance and cost estimation.

- *Provide feedback about user story*: (e.g. software team may require customer clarifications on requirements at any point of time).

2. Acceptance Testing & Releasing

The customer's approval is needed after completing each user story; the customer may review the working software and either accepts or rejects it.

The main acceptance testing processes carried out by the customer are:

- *Write Acceptance Test*: normally the customer is responsible for writing the test scenarios associated with each user story. These test scenarios describe what the user expects the system to do.
- *Modify Acceptance Test*: the customer may need to modify the provided test scenario.
- *Delete Acceptance Test*: acceptance tests can be deleted by the customer.
- *Change the status of an Acceptance Test to 'pass' or 'Fail'*: if the user story is implemented and integrated successfully, the customer should approve or reject the acceptance tests associated with the story.

In addition, some user stories might be completed early in the iteration and hence customer can run its associated acceptance tests before the end of the iteration. Therefore, software team may want to:

- *Request Running Acceptance Test*

This process might also be needed if acceptance testing failed and team members re-work on the corresponding user stories; customer needs to be contacted again for another acceptance testing after modifications are completed.

Furthermore, Releasing is a special process of the integration process where part of the software is delivered to the customer [12]. The associated release processes include:

- *Track release progress*

This process keeps the customer aware of the current status of the release and the amount of work done on the project.

III. TRADITIONAL METHODS OF CUSTOMER INVOLVEMENT IN DISTRIBUTED AGILE SOFTWARE PROJECTS

A. Using Customer Proxy

The good choice of a customer representative, who plays customer proxy role, minimizes the effect of the absence of an on-site customer [2]. Authors in [2] suggests assigning the role of customer proxy to a senior offshore developer with good domain knowledge. They believe that this involvement helps providing better understanding of the requirements. The use of customer proxy includes many disadvantages: it adds a level of complexity and sometimes customers face difficulties in reaching and communicating with customer's proxies.

B. Traveling to Customer Site

One of the current solutions to overcome the challenges resulting from the absence of an on-site customer is having the project manager travelled to the offshore team at specific points of project time (e.g. at the beginning of each iteration). This solution has been extensively discussed in the literature (e.g. [2]).

In their study, Rothman and Hastie [6] believe that the expense to meet in person is cheaper than the cost of not meeting. Case studies have shown that teams who travelled to meet customer succeeded more than teams who did not. The return on investment of the relatively small amount of money spent to bring the team together at the beginning of the project could be measured in the improvement of collaboration among the team members, shortened decision making cycles, fewer misunderstandings and rework, and higher quality work in less time to market.

Although this practice can bring benefits, sometimes it is simply not possible or is beyond the budget of the project. In addition, travelling to the customer will occur occasionally and can only support only high level coordination with customer. It is not practical to rely on this solution to manage the expected frequent communication between customer and development team during the project

C. Use of Rich Communication Technologies

Many studies suggest using rich communication technologies to minimize the challenges faced with communicating with customers during distributed agile projects [6]. This includes the use of:

Synchronous communication solutions such as video and phone conferencing tools. These are not useful if there is no possible overlapping time between software team and customer. In addition, these tools might be expensive for some teams.

Asynchronous communication solutions such as E-Mail Messaging. These are useful to send requests and file attachments, but still cannot be used as a central repository, as email content is distributed and separated. Managing and filtering emails becomes difficult as the team and project knowledge grow [13].

Many case studies reported difficulties using both synchronous or asynchronous communication [14-17]. Tool cost, planning for meetings, cultural and language differences, resolving technical issues, different holidays and different time zones, slow turnaround in communication and misunderstandings are all problems facing distributed agile teams while using these technologies.

As discussed above, all the current methods have problems associated with them. In addition to these problems, there is a common limitation that almost all distributed agile teams face when they have distant customer: team members may not be aware of how much their technical work needs customer involvement (e.g. change in code causes an acceptance test to fail). Although in some cases they know that customer involvement is necessary, they may forget to inform the customer or there might be a long time between the customer's need and the next possible meeting with the development team.

IV. TOOL SUPPORT FOR CUSTOMER INVOLVEMENT

A review of thirty-six agile and non-agile project management tools have been conducted to identify the current customer involvement mechanisms provided in these tools (shown in Table I). The table shows seven different mechanisms and states whether it is fully or partially supported by the tool. The review includes both commercial and open source tools. Most of the tools covered in the survey were designed to support agile development on the first class, while some of them support also project management in general.

1. Forums, Wikis, and Software Documentation

As online discussion is not always possible with distributed teams, some tools provide asynchronous collaboration with customers in form of forums, wikis, or product documentations with comments and voting features. These features are provided to keep the customer on the loop despite their availability and time zone differences.

2. Add Ideas or Requests

Most of the tools assume development teams are responsible for adding user stories and test cases into tools. They only allow customers submitting ideas or requests (11 tools fully support this and 4 tools partially support it by side online discussion or emails from customer).

3. Provide Feedback

Similar to ideas and requests posting, some tools (17 tools) allow customer to give feedback on release or specific user story. For instance, Mingle provides a feature called "Hold signoff", which creates a transition to allow customer to give feedback on user stories before they are considered 'Done'. JIRA collects customer satisfactions through simple workflow to provide feedback on resolved issues by both rating their level of satisfaction and adding comments.

4. Prioritize a User Story

About one-third of the studied tools allow prioritizing and/or voting mechanism either for customer submitted ideas or for available product requirements. In VersionOne, there is a voting platform that enables team members to capture, collaborate on and prioritize customer feature requests, and accordingly generate stories based on them. Planigle gives the customer the ability to view the backlog and give priority to items to be included in the next release.

5. Status Tracking

Some tools provide the ability to track status of user stories. This allows customer to keep track of the current project progress without the need to ask the team. The overall work progress and the status of backlog and release are available as a read-only feature to the customer by fourteen tools. This feature keeps the customer aware of the current status of the project.

6. Receive Notifications

Most of the tools allow customers to receive notifications whenever changes they are interested in are made. Usually this is not an automated process, and customers must manually subscribe in interested events or requests they want to be aware of. Subscriptions may be

done directly by the customer or by development team members in order to keep the customer in the loop.

7. Manage User Stories

Managing user stories includes adding, modifying and deleting user stories. Only 2 tools (Pivotal Tracker and IceScrum) fully support the management of user stories. In Pivotal Tracker, the story can be created by anyone who can make decisions about story requirements and acceptance criteria.

IceScrum allows customers to add, edit and delete stories in the sandbox. These stories must be refined by product owner before adding them into the product backlog. In AgileZen, customer can edit the status of stories. For example, story can be marked ready, indicating that it can be moved to the next phase. When customer encounters a problem with a story, they can mark it blocked, which can draw development team attention to the story.

TABLE I. REVIEW OF CUSTOMER INVOLVEMENT MECHANISMS IN AGILE PROJECT MANAGEMENT TOOLS

(● FULL SUPPORT, ○ PARTIAL SUPPORT)

Agile Tool	1		2	3	4	5	6	7	
	Forums /wikis	Software Documentations	Add Ideas/Requests	Provide Feedback	Prioritize a user story	Status tracking	Receive notifications	Add/Modify a user story	Delete user story
Rally			●				○		
Mingle			●	●					
VersionOne	●		●		●				
ScrumWorks	●		●	●		●	●		
TeamForge	●		●	●		●	●		
ExtremePlanner									
TargetProcess									
Pivotal Tracker			●	●	●		○	●	●
Scrum VSTS									
IceScrum			●	●	●	●	●	●	●
AgileWrap									
ScrumDesk				○					
Leankit									
DevSuite		●	●	●		●	●		
TinyPM	○	○	○				○	○	
Planigle					●				
Agilefant			●		●	●	○		
Axosoft	●	●	●	●	●		●		
AgileZen			○				●	○	○
Daily-Scrum			○						
Express									
JIRA		●	●	●	○	○	●	○	○
XPlanner	○	○							
BrixHQ									
Agile Bench									
SprintGround			○	○					
Planbox			●	●	●		●		
SpiraTeam				○	○				
Acunote									
Urban Turtle				○					
Wrike				●			●		
ActiveCollab	●	●		●			●		
Smartsheet		●					●		
Agilo							●		
Tuleap	●			●	●		●		
Odoo				●	●	●	●		

V. PROCESS SUPPORT NEEDED FOR CUSTOMER INVOLVEMENT

The available computer-based customer support mechanisms for distributed agile development team vary. Some tools do not provide customer role, and thus development team will manage customer involvement manually. In this case, they are likely to suffer from the limitations associated with the traditional management methods of customer involvement as mentioned in section III (i.e. they may not be aware of how their work needs customer involvement, they may forget to inform him, or there might be a long time between the customer's

participation need and the next possible meeting with the development team).

Although there is reasonable number of tool mechanisms supporting customer involvement, the current tools do not support most customer involvement processes identified in section II. In addition, those mechanisms that support customer involvement processes still have limitations. Table II summarizes to what extent the current tools support the main customer involvement processes. It also shows how current tools can better support the main customer involvement processes.

TABLE II. SUPPORT FOR CUSTOMER INVOLVEMENT PROCESSES

Customer Involvement Process	Support Mechanism in Current Tools	Proposed Support
Write/Modify a user story	- Add ideas /requests - Add/Modify a user story	Two of the current tools provide the basic process. Writing and modifying user stories can be supported by providing the necessary coordination (e.g. identify in which situations stories can be added or modified, identify who should be responsible for receiving the user story; notify the affected members).
Delete a user story	-Delete a user story	Two of the current tools provide the basic process. Deleting user stories can be supported by providing the necessary coordination (e.g. identify in which situations stories can be deleted, identify who should be affected by the deletion and notify the affected team members).
Prioritize a user story	- Prioritize a user story	Current tools allow one customer to prioritize user stories. This function can be improved by providing coordination mechanisms supporting voting of multiple customers, and adding a mechanism to manage possible conflicts between them.
Provide feedback about user story	- Provide feedback	Current tools provide the basic process. It can be supported by notifying corresponding team members about a new feedback posted.
Write/ Modify Acceptance Test	- Not supported	Current tools do not support these two processes entirely. Two new functions can be proposed to allow customer to add/modify acceptance tests. The functions can also: - identify in which situations customer can add/modify acceptance tests. - identify who should be affected by the addition/modification and notify the affected team members.
Delete Acceptance Test	- Not supported	Current tools do not support this process entirely. A new function can be proposed to allow customer to delete acceptance tests. The function can also: - identify in which situations acceptance tests can be deleted. - identify who should be affected by the deletion and notify the affected team members.
Change the status of an Acceptance Test to 'Pass'/'Fail'	- Not supported	Current tools do not support this process entirely. A new function can be proposed to allow customer change status of acceptance tests. The function can also: - identify in which situations status of acceptance tests can be changed. - identify who should be affected by changing status of an acceptance test and notify the affected team members.
Request Running Acceptance Test	- Not supported	Current tools do not support this process entirely. Automating such process can save customer's time. He can be involved when required only.
Track release Progress	- Status tracking - Receive notifications	Current tools allow customer to view the progress but do not automatically discover any potential delay (e.g. in delivering a release) and inform customer in advance. This can be done in a new function.

VI. CONCLUSIONS

This research is about helping customers be more involved in globally distributed agile projects. The traditional approaches have been reviewed in this paper. The review revealed that there is still a need for more support for customer involvement. Thus, more formal methods appeared through the use of project management tools.

The tool support provided so far is still weak. The review of tools shows that all customer involvement processes are either entirely absent in the current tools or limited.

This paper proposed a set of computer-based functionalities that can help support customer involvement in globally distributed agile projects. The new functionalities do not replace the need for informal communication between customer and team members as it is always important to be close to customer. However, not all involvements require synchronous communications. In addition, depending completely on humans to identify and coordinate how technical changes affect customer involvement is unrealistic.

Future work will continue as follow: first, we will design a computer-based model capable of providing the process support proposed in this paper. Second, we will evaluate the approach through implementing a computer system and deliver it to distributed agile project management teams for assessment.

REFERENCES

- [1] K. Beck, M. Beedle, A. v. Bennekum, A. Cockburn, W. Cunningham, M. Fowler, *et al.* (2001, September 2014). *Manifesto for Agile Software Development*. Available: www.agilemanifesto.org
- [2] M. Kajko-Mattsson, G. Azizyan, and M. K. Magarian, "Classes of Distributed Agile Development Problems," presented at the Proceedings of the 2010 Agile Conference, 2010.
- [3] I. Sommerville, *Software Engineering*: Addison-Wesley Publishing Company, 2010.
- [4] I. A. N. Inayat, Muhammad; Inayat, Zubaria, "Successful Product-based Agile Software Development without Onsite Customer," *International Journal of Software Engineering & Its Applications*, vol. 6, pp. 1-14, April, 2012 2012.
- [5] S. Dorairaj and J. Noble, "Agile Software Development with Distributed Teams: Agility, Distribution and Trust," in *Agile Conference (AGILE)*, 2013, 2013, pp. 1-10.
- [6] J. Rothman and S. Hastie, "Lessons Learned from Leading Workshops about Geographically Distributed Agile Teams," *IEEE Softw.*, vol. 30, pp. 7-10, 2013.
- [7] M. Korkala, M. Pikkarainen, and K. Conboy, "Distributed Agile Development: A Case Study of Customer Communication Challenges," in *Agile Processes in Software Engineering and Extreme Programming*. vol. 31, P. Abrahamsson, M. Marchesi, and F. Maurer, Eds., ed: Springer Berlin Heidelberg, 2009, pp. 161-167.
- [8] M. Korkala, M. Pikkarainen, and K. Conboy, "Combining Agile and Traditional: Customer Communication in Distributed Environment," in *Agility Across Time and Space*, D. Šmite, N. B. Moe, and P. J. Ågerfalk, Eds., ed: Springer Berlin Heidelberg, 2010, pp. 201-216.
- [9] J. Koskela and P. Abrahamsson, "On-site customer in an XP project: empirical results from a case study," in *Software Process Improvement*, ed: Springer, 2004, pp. 1-11.
- [10] S. Alyahya, W. K. Ivins, and W. Gray, "Raising the Awareness of Development Progress in Distributed Agile Projects," *Journal of Software*, vol. 8, pp. 3066-3081, 2013.
- [11] L. Maruping, "Implementing Extreme Programming in Distributed Software Project Teams: Strategies and Challenges," in *Agility Across Time and Space*, D. Šmite, N. B. Moe, and P. J. Ågerfalk, Eds., ed: Springer Berlin Heidelberg, 2010, pp. 11-30.
- [12] S. Alyahya, W. K. Ivins, and W. A. Gray, "A Holistic Approach to Developing a Progress Tracking System for Distributed Agile Teams," pp. 503-512, 2012.
- [13] S. H. Rayhan and N. Haque, "Incremental Adoption of Scrum for Successful Delivery of an IT Project in a Remote Setup," in *Agile, 2008. AGILE '08. Conference*, 2008, pp. 351-355.
- [14] S. Alyahya, "A computer-based holistic approach to managing progress of distributed agile teams," PhD Thesis, School of Computer Science and Informatics, Cardiff University, 2013.
- [15] M. Paasivaara, S. Durasiewicz, and C. Lassenius, "Using Scrum in Distributed Agile Development: A Multiple Case Study," in *Global Software Engineering, 2009. ICGSE 2009. Fourth IEEE International Conference on*, 2009, pp. 195-204.
- [16] E. Therrien, "Overcoming the Challenges of Building a Distributed Agile Organization," in *Agile, 2008. AGILE '08. Conference*, 2008, pp. 368-372.
- [17] W. Williams and M. Stout, "Colossal, Scattered, and Chaotic (Planning with a Large Distributed Team)," in *Agile, 2008. AGILE '08. Conference*, 2008, pp. 356-361.